Práctica de JavaCC

Mini-REXX: traducción a una máquina de pila

Curso 2026–2027 ▪ Procesadores de Lenguajes

1. Enunciado

Se trata de utilizar **JavaCC** para construir un compilador que traduzca un lenguaje de programación de alto nivel, **Mini-REXX**, a instrucciones de la **máquina de pila abstracta** utilizada en las prácticas de la asignatura. El lenguaje fuente está inspirado en REXX, un lenguaje real concebido para la automatización y la escritura de procedimientos. Sin embargo, las reglas que se describen a continuación **definen íntegramente Mini-REXX**: no es necesario conocer REXX ni reproducir sus características no mencionadas.

El lenguaje no exige declaraciones de variables. Se trabaja con **valores enteros**; el valor cero representa falso y cualquier otro valor representa verdadero. Las cadenas solo pueden aparecer en las instrucciones de salida.

Además de las expresiones aritméticas clásicas, este lenguaje cuenta con la expresión `CHOOSE(condición, expresión1, expresión2)`, donde se evalúa primero la condición, si es verdadera, se evalúa y devuelve la primera expresión, en caso contrario, se evalúa y devuelve la segunda. **No se evalúa la rama descartada.** Esta construcción puede anidarse y aparecer dentro de expresiones aritméticas o de las condiciones de otras instrucciones.

1.1. Construcciones del lenguaje

- **Asignación:** `x = expresión`.
- **Entrada:** `PULL x`, que lee un entero y lo almacena en la variable indicada.
- **Salida:** `SAY elemento, elemento, ...`, con al menos un elemento; cada elemento es una cadena literal o una expresión aritmética. Se imprime la concatenación de los elementos *sin añadir espacios automáticamente*, seguida de un salto de línea.
- **Condicional:** `IF condición THEN DO ... END` y, opcionalmente, `ELSE DO ... END`. Las ramas siempre son bloques `DO...END`, incluso si contienen una sola instrucción.
- **Selección múltiple:** `SELECT` con una o más cláusulas `WHEN condición THEN DO ... END`, una cláusula `OTHERWISE DO ... END` opcional y un `END` final. Solo se ejecuta la primera rama cuya condición sea verdadera.
- **Bucle preevaluado:** `DO WHILE condición ... END`.
- **Bucle postevaluado:** `DO UNTIL condición ... END`; su cuerpo se ejecuta al menos una vez, y se abandona cuando la condición es verdadera.
- **Bucle con contador:** `DO i = expresión TO expresión [BY [ADDING|SUBTRACTING] NUM] ... END`, con incremento por defecto de 1.

1.2. Semántica particular del bucle con contador

El valor inicial se calcula **una sola vez**. El límite indicado después de `TO` se **vuelve a evaluar antes de cada iteración**. Se ejecuta el cuerpo mientras el valor del contador sea *menor o igual* que dicho límite; después de cada iteración, el contador se incrementa en el valor indicado por

`BY` (o por `BY ADDING`) o en 1 si se omite. Si se quiere decrementar el valor del contador hay que utilizar la construcción `BY SUBTRACTING`. El valor de actualización del `BY` tiene que ser un entero distinto de cero. **Si se usará el valor cero, habría que emitir un mensaje de error**. Esta es la única comprobación semántica que hace falta hacer en esta práctica. Salvo esto, podemos asumir que el programa de entrada es correcto. El cuerpo de un bucle con contador puede modificar otras variables, incluido el límite, si este fuera un identificar, en vez de una expresión. Ni el `BY ADDING`, ni el `BY SUBTRACTING` existen en REXX, el lenguaje en el que está inspirada esta práctica.

Esta semántica se ha elegido expresamente para **evitar variables auxiliares en el código generado**; difiere de la semántica del bucle con contador en REXX completo, que evalúa el límite una sola vez al entrar en el bucle.

1.3. Expresiones y condiciones

Se admiten los operadores aritméticos `+`, `-`, `*`, `/`, además de los signos unarios `+` y `-` y los paréntesis. La multiplicación y la división tienen mayor precedencia que la suma y la resta; los operadores binarios de un mismo nivel son asociativos por la izquierda. La división es entera y trunca hacia cero. No se define el resultado de una división entre cero.

Una condición es una expresión aritmética, opcionalmente seguida de **un** operador relacional y de otra expresión: `=`, `<>`, `<`, `>`, `<=` o `>=`. Cuando se omite la comparación, el propio resultado de la expresión se interpreta como valor lógico. `CHOOSE` puede aparecer en cualquiera de las expresiones y admite como primer argumento esta misma forma de condición; sus otras dos ramas deben ser expresiones aritméticas. La expresión `CHOOSE` no existe en REXX.

2. Reglas léxicas

Elemento	Regla
`ID`	Una letra ASCII o guion bajo seguidos de cero o más letras ASCII, dígitos o guiones bajos. No puede comenzar por dígito ni coincidir con una palabra reservada. Ejemplos: `a`, `TOTAL_1`, `_x`.
`NUM`	Una o más cifras decimales.
`CAD`	Secuencia de caracteres entre comillas simples o entre comillas dobles. No puede contener un salto de línea ni las comillas de su mismo tipo. No hay secuencias de escape.
Comentarios	Comienzan con `/*` y terminan con `*/`; pueden ocupar varias líneas y **no** se anidan.
Palabras reservadas	`PULL, SAY, IF, THEN, ELSE, DO, END, WHILE, UNTIL, TO, BY, SELECT, WHEN, OTHERWISE, CHOOSE, ADDING, SUBTRACTING`.
Espacios y separadores	Se ignoran espacios, tabuladores, cambios de línea y el carácter `;`. Así, los separadores entre sentencias son opcionales: las propias construcciones sintácticas delimitan las instrucciones.
Mayúsculas	Las palabras reservadas y los identificadores no distinguen mayúsculas de minúsculas: `total`, `TOTAL` y `ToTaL` son la misma variable.

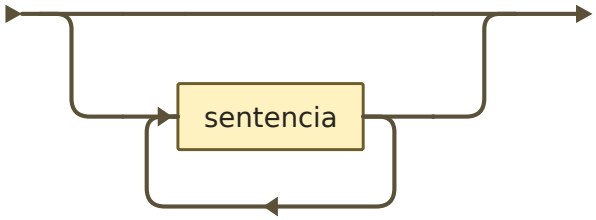
Ejemplos léxicos: `"Hola 'mundo'"` y `'Hola "mundo"'` son cadenas válidas, pero `"Hola'` no lo es. Los literales `10.5` y `0x10` no forman parte del lenguaje. Una cadena solo es válida como elemento de `SAY`, no como operando aritmético ni como variable.

3. Diagramas de sintaxis

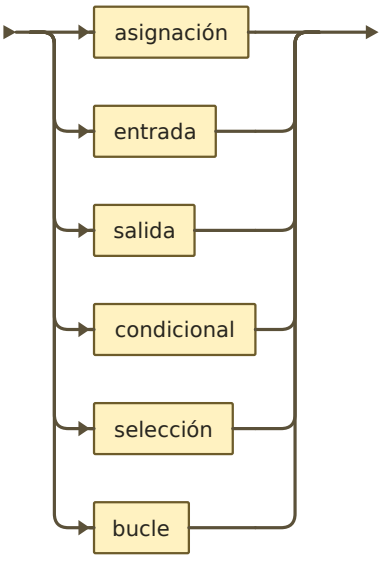
Los diagramas siguientes utilizan rectángulos redondeados para **terminales** y rectángulos rectos para **no terminales**. Las ramificaciones representan alternativas; las derivaciones de retorno indican repetición. `sentencias` equivale a cero o más sentencias. Los diagramas representan el lenguaje fuente, no las instrucciones de la máquina destino.

3.1. Estructura general y entrada/salida

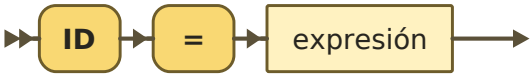
programa:



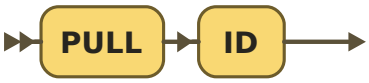
sentencia:



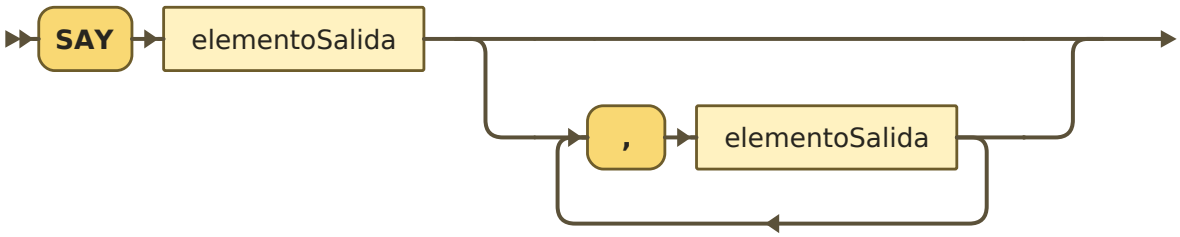
asignacion:



entrada:



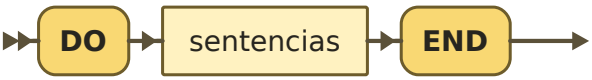
salida:



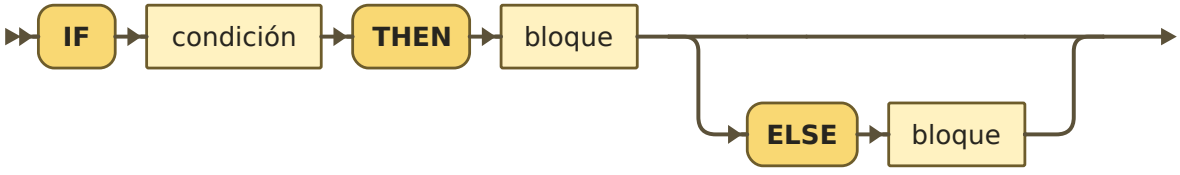
elementoSalida se define como `CAD` o `expresión`.

3.2. Bloques y selección

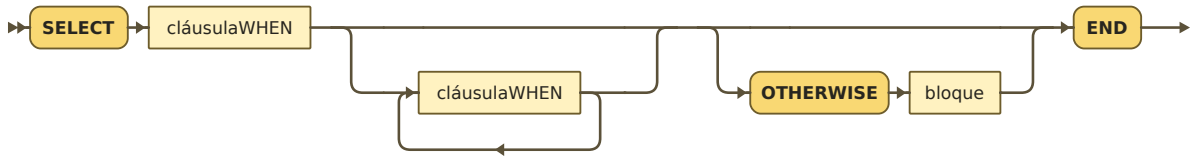
bloque:



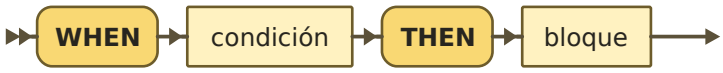
condicional:



seleccion:



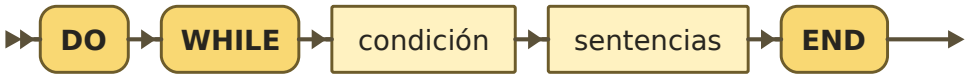
clausulaWHEN:



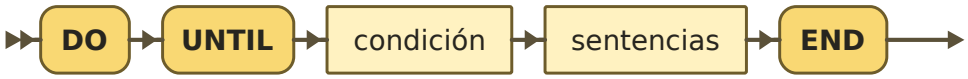
Los bloques empleados por `IF`, `WHEN` y `OTHERWISE` son siempre del tipo `DO sentencias END`. En `SELECT` es obligatorio incluir al menos una cláusula `WHEN`.

3.3. Los tres tipos de bucles

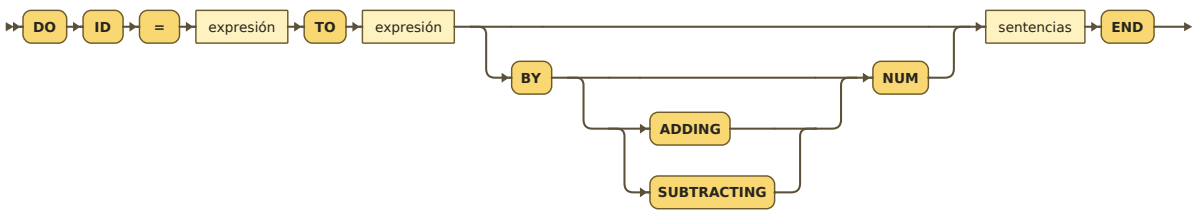
bucleWhile:



bucleUntil:



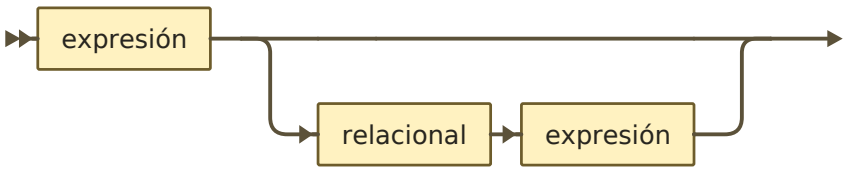
bucleContador:



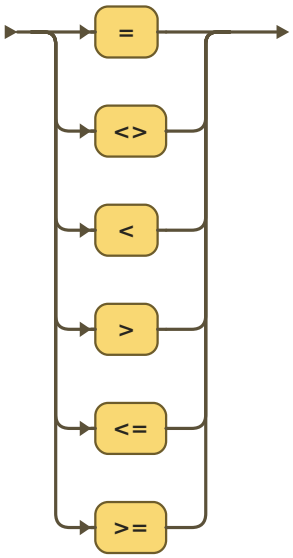
bucle se define como la alternativa entre estas tres construcciones. La `expresion` del límite `TO` se recalcula en cada comprobación. Si no hay una especificación `BY` se asume un incremento de 1. Si se utiliza una especificación `BY SUBTRACTING` el valor de `NUM` se resta del contador en cada iteración.

3.4. Condiciones y expresiones

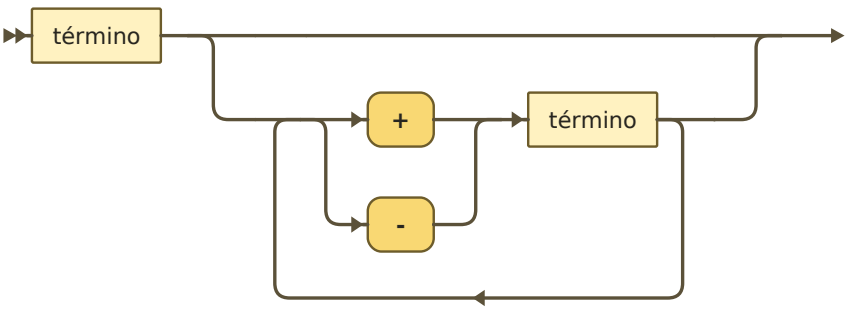
condicion:



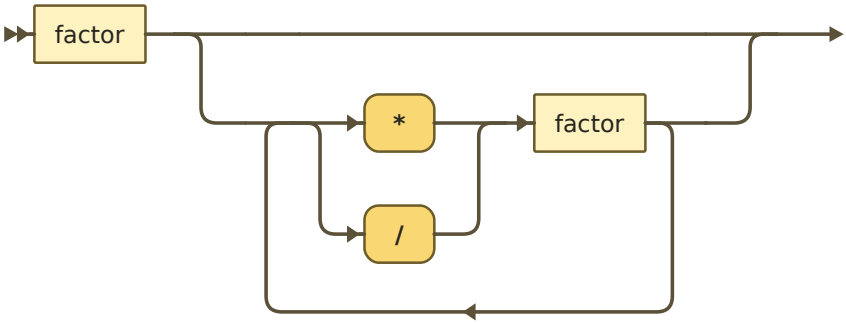
relacional:



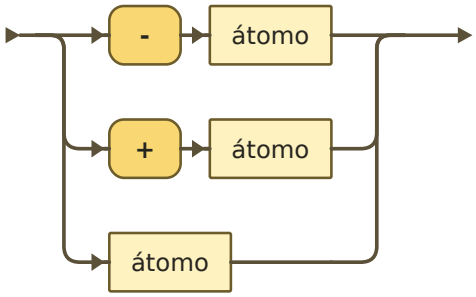
expresion:



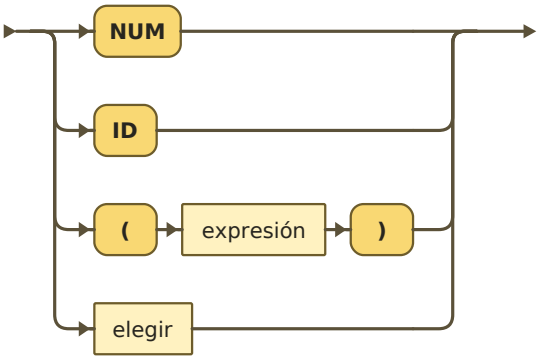
termino:



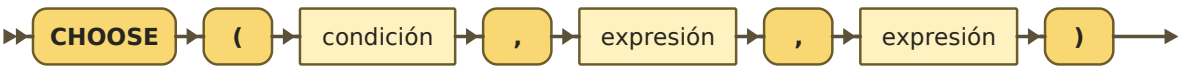
factor:



atomo:



elegir:



4. Máquina de pila de destino

Se utilizará el lenguaje intermedio habitual, sin introducir nuevas instrucciones. Estas son las operaciones necesarias; a y b se refieren a los dos operandos superiores de la pila, con b en el tope.

Instrucción	Efecto
`mete n`	Introduce el entero `n` en la pila.
`valord x`	Introduce el valor almacenado en la variable `x`.
`valori x`	Introduce una referencia a la variable `x` para una asignación posterior.
`asigna`	Retira el valor del tope y la referencia inmediatamente inferior y almacena el valor en la variable correspondiente.
`add, sub, mul, div`	Sustituyen los dos operandos por su suma, diferencia $a-b$, producto o cociente entero a/b.
`esigual, esmenor, esmayor`	Sustituyen a, b por 1 si la comparación $a = b$, $a < b$ o $a > b$ se cumple, o por 0 si no.
`not`	Sustituye el tope por 1 si es 0; en otro caso, por 0.
`swap`	Intercambia los dos elementos superiores.
`lee x`	Lee un entero de la entrada de ejecución y lo almacena en `x`.
`metecad cadena`	Introduce en la pila una referencia a la cadena literal indicada.
`print n`	Extrae `n` elementos e imprime su contenido, respetando el orden en que fueron apilados, sin separadores automáticos; añade un salto de línea.
`vea L`	Salta incondicionalmente a la etiqueta `L`.
`sifalsovea L`	Extrae el tope y salta si es cero.
`siciertovea L`	Extrae el tope y salta si es distinto de cero.
`LBLk:`	Define una etiqueta como destino de saltos. Los nombres concretos pueden variar.

Restricción esencial: el compilador *no* podrá introducir variables auxiliares en el código intermedio. Solo podrán aparecer en `valori`, `valord` y `lee` las variables escritas por el programador, sin distinguir mayúsculas y minúsculas. Sí se permite crear las etiquetas necesarias y utilizar búferes de código durante la compilación.

5. Ejemplos de programas y traducciones

En todos los ejemplos, la numeración de etiquetas es orientativa. *Salida intermedia* significa el texto emitido por el compilador; *salida de ejecución* significa lo que imprimiría la máquina de pila al ejecutar ese texto.

5.1. Ejemplo 1. Operaciones aritméticas

Programa fuente:

```
/* Precedencia y paréntesis */
 a = 5 + 3 * 2
 b = (a - 1) / 2
 say "a=", a, "; b=", b
```

Salida intermedia esperada:

```
    valori a
    mete 5
    mete 3
    mete 2
    mul
    add
    asigna
    valori b
    valord a
    mete 1
    sub
    mete 2
    div
    asigna
    metecad "a="
    valord a
    metecad "; b="
    valord b
    print 4
```

5.2. Ejemplo 2. Expresiones condicionales, dos ramas

Programa fuente:

```
x = 9; y = 4
r1 = choose(x > y, x * 2, y + 5)    say "r1=", r1
r2 = choose(x < y,
           x * 2,
           y + 5)
say "r1+r2=", r1+r2, "; r1-r2=", r1-r2
```

Salida intermedia esperada:

```
    valori x
    mete 9
    asigna
    valori y
    mete 4
    asigna
    valori r1
    valord x
    valord y
    esmayor
    sifalsovea LBL0
    valord x
    mete 2
    mul
    vea LBL1
LBL0:
    valord y
    mete 5
    add
LBL1:
    asigna
    metecad "r1="
    valord r1
    print 2
    valori r2
    valord x
    valord y
```

```
    esmenor
    sifalsovea LBL2
    valord x
    mete 2
    mul
    vea LBL3
LBL2:
    valord y
    mete 5
    add
LBL3:
    asigna
    metecad "r1+r2="
    valord r1
    valord r2
    add
    metecad "; r1-r2="
    valord r1
    valord r2
    sub
    print 4
```

Cada `CHOOSE` debe dejar **exactamente un valor** sobre la pila. El compilador genera un salto alrededor de la rama descartada, no una evaluación previa de las dos ramas. Esto permite expresiones anidadas o combinaciones como `4 + CHOOSE(x > 0, x, 0) * 3`.

5.3. Ejemplo 3. Expresiones condicionales anidadas

Programa fuente:

```
x = 2
y = 7
z = 10 + choose(x > y, 3, choose(y >= 7, y * 2, 0)) * 3
say "z=", z
```

Salida intermedia esperada:

```
    valori x
    mete 2
    asigna
    valori y
    mete 7
    asigna
    valori z
    mete 10
    valord x
    valord y
    esmayor
    sifalsovea LBL0
    mete 3
    vea LBL1
LBL0:
    valord y
    mete 7
    esmenor
    not
    sifalsovea LBL2
    valord y
    mete 2
    mul
    vea LBL3
LBL2:
```

```
    mete 0
LBL3:
LBL1:
    mete 3
    mul
    add
    asigna
    metecad "z="
    valord z
    print 2
```

5.4. Ejemplo 4. Sentencia `if-then-else`

Programa fuente:

```
x = 4
if x > 5 then do
   say 'GRANDE'
end
else do
   say "PEQUENO"
end
```

Salida intermedia esperada:

```
    valori x
    mete 4
    asigna
    valord x
    mete 5
    esmayor
    sifalsovea LBL0
    metecad 'GRANDE'
    print 1
    vea LBL1
LBL0:
    metecad "PEQUENO"
    print 1
LBL1:
```

En una sentencia condicional la parte `else` es opcional.

5.5. Ejemplo 5. Sentencia `select`

Programa fuente:

```
n = 17
select
    when n < 10 then do say "INFANTIL" end
    when n < 18 then do say "JUVENIL" end
    otherwise do say "ADULTO" end
end
```

Salida intermedia esperada:

```
    valori n
    mete 17
    asigna
    valord n
```

```
    mete 10
    esmenor
    sifalsovea LBL1
    metecad "INFANTIL"
    print 1
    vea LBL0
LBL1:
    valord n
    mete 18
    esmenor
    sifalsovea LBL2
    metecad "JUVENIL"
    print 1
    vea LBL0
LBL2:
    metecad "ADULTO"
    print 1
LBL0:
```

En las sentencias `select` la cláusula `otherwise` es opcional.

5.6. Ejemplo 6. Bucle `DO WHILE`

Programa fuente:

```
n = 3
 do while n > 0
    say n
    n = n - 1
 end
```

Salida intermedia esperada:

```
    valori n
    mete 3
    asigna
LBL0:
    valord n
    mete 0
    esmayor
    sifalsovea LBL1
    valord n
    print 1
    valori n
    valord n
    mete 1
    sub
    asigna
    vea LBL0
LBL1:
```

5.7. Ejemplo 7. Bucle `DO UNTIL`

Programa fuente:

```
n = 0
 do until n = 0
    say "UNA VEZ"
 end
```

Salida intermedia esperada:

```
    valori n
    mete 0
    asigna
    vea LBL1
LBL0:
    valord n
    mete 0
    esigual
    siciertovea LBL2
LBL1:
    metecad "UNA VEZ"
    print 1
    vea LBL0
LBL2:
```

A diferencia de `DO WHILE`, `DO UNTIL` evalúa su condición *después* de ejecutar el cuerpo. Para poder emitir el código en el orden con que nos encontramos los elementos en el programa de entrada, nos vemos forzados a comenzar con un salto incondicional al cuerpo del bucle, para luego volver al código donde se evalúa la condición.

5.8. Ejemplo 8. Bucle con paso sustractivo

Programa fuente:

```
s = 0
do i = 5 to 1 by subtracting 2
    s = s + i
end
say "SUMA: ", s
```

Salida intermedia esperada:

```
    valori s
    mete 0
    asigna
    valori i
    mete 5
    asigna
LBL0:
    valord i
    mete 1
    esmayor
    siciertovea LBL1
    valori s
    valord s
    valord i
    add
    asigna
    valori i
    valord i
    mete 2
    sub
    asigna
    vea LBL0
LBL1:
    metecad "SUMA: "
    valord s
    print 2
```

6. Notas e indicaciones

1. Es preciso convertir los diagramas en producciones adecuadas para JavaCC. Se recomienda factorizar y aprovechar los operadores `?`, `*` y `+`. **No se permite utilizar `LOOKAHEAD`** para resolver conflictos de la gramática; las producciones deben evitar advertencias de ambigüedad.
2. El traductor debe aceptar el código fuente desde **entrada estándar** si no recibe argumentos y desde el **fichero** pasado como primer argumento en caso contrario. El código de máquina se escribe en la salida estándar.
3. No se permite generar **ninguna variable temporal** en el lenguaje intermedio. Sí pueden utilizarse variables y objetos de Java dentro del compilador, así como almacenarse temporalmente fragmentos del código generado cuando sea necesario (por ejemplo, para `DO UNTIL`).
4. `CHOOSE` siempre debe evaluar solo una rama y dejar un único entero en el tope de la pila. En particular, deben funcionar las expresiones anidadas y los contextos en los que se combina con multiplicaciones, sumas o comparaciones.
5. Se valorarán la claridad del código JavaCC, el tratamiento correcto de comentarios y palabras reservadas, la generación coherente de etiquetas únicas y la calidad de las pruebas aportadas.
6. Se penalizará resolver en Java cosas que se pueden hacer en directamente en JavaCC.
7. Que el programa no emita exactamente las mismas etiquetas que las mostradas en los ejemplos no es un problema por sí mismo siempre que la semántica de los saltos sea correcta para la instrucción que se está traduciendo.

7. Nota histórica

REXX fue diseñado por Mike Cowlishaw en IBM a finales de los años setenta como un lenguaje de procedimientos legible y flexible. Su estilo de instrucciones, como `SAY`, `PULL`, `DO` y `SELECT`, resulta especialmente adecuado para explorar la traducción de estructuras de control. En esta práctica se utiliza una versión simplificada y se incorpora deliberadamente `CHOOSE`, una expresión condicional propia de Mini-REXX que no pretende reproducir una instrucción estándar del REXX original.